**What is an Abstract Class**

An abstract class is a class that cannot be instantiated directly. Instead, it acts as a base that other classes must extend. Its main purpose is to define common structure and behavior that related classes share, while also enforcing certain rules through abstract members.

**Key Characteristics of Abstract Classes**

- **Cannot be instantiated**: Abstract classes exist only to be inherited, not to create objects directly.
- **May contain abstract methods**: These are methods without implementation that must be defined by subclasses.
- **Can contain concrete methods**: Unlike interfaces, abstract classes can also provide fully implemented methods to be reused by child classes.
- **Support fields and properties**: Abstract classes can declare fields, properties, and even constructors.
- **Enable partial abstraction**: A class can provide some implemented functionality while leaving other aspects abstract.

**Why Use Abstract Classes?**

Abstract classes are most useful when you want to enforce a shared contract across multiple classes, but also need to include common functionality. They provide a balance between strict enforcement and flexibility. This makes them ideal for situations where different subclasses must implement specific behavior while still sharing a base of predefined logic.

**Abstract Classes vs. Interfaces**

Abstract classes and interfaces both define contracts, but they serve slightly different purposes. Interfaces focus purely on defining behaviors without implementation. Abstract classes, on the other hand, allow partial implementation and shared state. This makes abstract classes more suitable when there is a clear inheritance relationship, while interfaces are better for defining roles that can be applied across unrelated classes.

**Advantages of Abstract Classes**

- Promote code reusability by allowing shared functionality in the base class.
- Enforce consistency across derived classes through abstract members.
- Offer flexibility by mixing implemented and unimplemented methods.
- Support encapsulation of common state and behavior.

**When to Use Abstract Classes**

- When multiple related classes share a common base but also require specific implementations.

- When you need to define default behavior along with mandatory behaviors.
- When you want to model an "is-a" relationship between a base type and its subtypes.